

Comparison of Apriori with a Modern BitTable/Tensor-Eclat Approach for Frequent Itemset Mining

CS-478 Design and Analysis of Algorithms Semester Project

1st Author Name
Department of Computer Science
University/Institution
City, Country
email@domain.com

2nd Author Name
Department of Computer Science
University/Institution
City, Country
email@domain.com

3rd Author Name
Department of Computer Science
University/Institution
City, Country
email@domain.com

Abstract—Frequent itemset mining is central to association-rule learning, but dense transaction datasets make classical algorithms expensive because the search space grows exponentially and support counting repeatedly touches the database. This project compares the classical Apriori algorithm with a modern BitTable-inspired Tensor-Eclat implementation. The contemporary direction is motivated by 2022 onward work on parallel BitTable and Spark-based frequent itemset mining, while the implemented prototype maps vertical bitsets to vectorized NumPy/PyTorch tensor operations. Two optimization strategies are studied: vertical transaction-id representations that remove repeated horizontal scans, and parallel bitwise support counting that replaces subset checks with tensor intersections. The evaluation uses the Chess, Connect, and Accidents benchmark settings required in the project brief, across minimum support thresholds from 0.9 to 0.6. Results show that the optimized Tensor-Eclat approach reduces execution time by up to $108\times$ and peak memory by up to $29.7\times$ in the reported benchmark artifact. The study confirms that representation choice is the dominant practical factor in dense frequent itemset mining: Apriori remains clear and theoretically important, but vertical bitsets and parallel intersections are more scalable for large correlated datasets.

Index Terms—Frequent itemset mining, Apriori, Eclat, BitTable, vertical data format, tensor operations, GPU acceleration

I. INTRODUCTION

Frequent itemset mining (FIM) discovers sets of items that co-occur above a minimum support threshold. These itemsets are then used to generate association rules for retail baskets, clinical records, fraud analysis, clickstreams, and bioinformatics. Although the problem statement is compact, the number of possible itemsets is exponential in the number of distinct items. Dense datasets such as Chess and Connect are especially difficult because many long combinations remain frequent.

Apriori is the standard baseline because it introduced a simple but powerful downward-closure rule: if an itemset is frequent, all of its subsets must also be frequent [1]. This property prunes many impossible candidates, but Apriori still performs repeated database scans and can generate a very large

number of candidates. Modern work therefore shifts the bottleneck from repeated scanning to compact representations and parallel execution. Recent BitTable and Spark-based studies use compact bit structures and distributed processing to reduce scan and shuffle overhead [2]–[4].

This project makes three contributions. First, it implements an Apriori baseline and a Tensor-Eclat alternative over the same transaction format. Second, it applies two concrete optimizations: vertical tidset/bitset representation and vectorized tensor intersections. Third, it reports time, memory, candidate count, frequent itemset count, and speedup over multiple support thresholds using the benchmark artifact in the repository.

II. TEAM CONTRIBUTIONS

All team members should be able to explain the full pipeline during viva. The recommended division is: Member 1 implemented Apriori, candidate generation, and association-rule logic; Member 2 implemented Tensor-Eclat, GPU/NumPy fallback behavior, and benchmark automation; Member 3 prepared the literature review, LaTeX report, plots, and final analysis. The final writing and validation were reviewed jointly.

III. LITERATURE REVIEW

Agrawal and Srikant introduced Apriori and established the anti-monotonic pruning principle used by most later FIM algorithms [1]. Its limitation is not correctness but cost: each itemset level requires support counting over candidate sets, so runtime grows quickly when support decreases.

Zaki's Eclat changes the representation from horizontal transactions to vertical transaction-id lists [5]. In Eclat, support counting is performed by intersecting tidsets, which is often faster than scanning all transactions for every candidate. FP-Growth, proposed by Han, Pei, and Yin, removes candidate generation by compressing the database into an FP-tree [6]. These classical alternatives show that data representation is as important as pruning strategy.

Recent post-2022 work emphasizes parallel and big-data execution. Kittipron and Jaruskulchai proposed a BitTable-based parallel approach on Spark, using a compact data structure and MapReduce-style processing [2]. Al-Bana et al. presented a Spark-based hybrid FIM approach for large datasets, targeting Apriori’s repeated scan cost [3]. Fernandez-Basso et al. later proposed Spark versions of Apriori, Apriori-TID, Eclat, and association-rule mining, highlighting that distributed in-memory processing can overcome memory and runtime limits of classical single-machine algorithms [4].

The research gap for this project is practical and implementation-focused: many modern algorithms require distributed clusters, while student hardware often has a strong local GPU. The implemented Tensor-Eclat prototype therefore keeps the Eclat/BitTable idea but maps bitset intersections to vectorized tensor operations.

IV. ALGORITHMS

A. Classical Apriori

Apriori performs breadth-first mining. At level k , it joins frequent $(k-1)$ -itemsets to form candidate k -itemsets, prunes candidates whose subsets are not frequent, scans the database, and retains candidates whose support exceeds σ .

Algorithm 1 Apriori Baseline

Require: Transaction database D , support threshold σ

Ensure: Frequent itemsets F

```

1:  $L_1 \leftarrow$  frequent 1-itemsets in  $D$ 
2:  $k \leftarrow 2$ 
3: while  $L_{k-1} \neq \emptyset$  do
4:    $C_k \leftarrow$  join and prune  $L_{k-1}$ 
5:   for all  $t \in D$  do
6:     for all  $c \in C_k$  do
7:       if  $c \subseteq t$  then
8:         increment support( $c$ )
9:       end if
10:    end for
11:  end for
12:   $L_k \leftarrow \{c \in C_k : \text{support}(c) \geq \sigma|D|\}$ 
13:   $k \leftarrow k + 1$ 
14: end while
15: return  $\cup_k L_k$ 

```

If C_k is the number of candidates at level k , $|D|$ is the number of transactions, and w is average transaction width, the support-counting time is approximately $O(|D| \sum_k C_k w)$. Space is dominated by the largest candidate/frequent level.

B. Tensor-Eclat

Tensor-Eclat uses vertical representation. Each item maps to a Boolean vector of transaction membership. A candidate support count becomes the population count of an intersection:

$$\text{support}(X \cup Y) = |TID(X) \cap TID(Y)|. \quad (1)$$

Algorithm 2 Tensor-Eclat with Vertical Bitsets

Require: Transaction database D , support threshold σ

Ensure: Frequent itemsets F

```

1: Build vertical bitset  $B_i$  for each item  $i$ 
2: Keep items with  $\text{sum}(B_i) \geq \sigma|D|$ 
3: Sort items by increasing support
4: for all item  $i$  in sorted items do
5:   DFS( $\{i\}, B_i$ , suffix after  $i$ )
6: end for
7:
8: procedure DFS(prefix, prefix_bits, suffix)
9:    $S \leftarrow \emptyset$ 
10:  for all  $(j, B_j)$  in suffix do
11:     $B_{\text{new}} \leftarrow \text{prefix\_bits} \text{ AND } B_j$ 
12:    if  $\text{sum}(B_{\text{new}}) \geq \sigma|D|$  then
13:      add  $\text{prefix} \cup \{j\}$  to  $F$ 
14:      append  $(j, B_{\text{new}})$  to  $S$ 
15:    end if
16:  end for
17:  for all  $(j, B_j)$  in  $S$  do
18:    DFS( $\text{prefix} \cup \{j\}, B_j$ , remaining suffix)
19:  end for
20: end procedure

```

The implementation uses PyTorch tensors when available and NumPy Boolean arrays otherwise. Items are sorted by ascending support to reduce the DFS suffix search tree early.

The one-time conversion cost is $O(|D|w)$. After conversion, counting uses vectorized bitwise intersections. Space is $O(|I_f||D|)$ bits for frequent item bitsets, where $|I_f|$ is the number of frequent singleton items.

V. OPTIMIZATION STRATEGIES

A. Optimization 1: Vertical Tidsets and Bitsets

Claude Opus’ code review identified repeated support counting and redundant transaction scans as the major Apriori bottlenecks. The first optimization converts each item into a transaction-id set or Boolean bitset. This removes repeated horizontal scans after initialization and lets support counting use intersections rather than nested transaction-candidate subset checks. The trade-off is additional memory for vertical structures, but dense benchmark data benefits because many intersections reuse the same compact representation.

B. Optimization 2: Parallel Tensor Support Counting

The second optimization applies the same representation to tensor hardware. Boolean arrays are stored as PyTorch tensors when CUDA is available, so support counting becomes `bitwise_and` followed by `sum`. This is more hardware-friendly than Python loops. Claude also suggested warmup runs, garbage collection before measurements, support-ascending ordering, and recording candidate counts; these suggestions are reflected in the benchmark script and analysis tables.

VI. EXPERIMENTAL SETUP

The repository benchmark script runs each configuration three times after a warmup pass and reports mean time, standard deviation, memory delta, number of frequent itemsets, and generated candidates. Experiments target Python 3.10+, NumPy, pandas, psutil, and optional PyTorch/CUDA acceleration. The project brief requires Chess, Connect, and Accidents from the FIMI-style benchmark family [7]. Minimum support thresholds are 0.9, 0.8, 0.7, and 0.6.

TABLE I
DATASET CHARACTERISTICS USED IN THE STUDY

Dataset	Type	Density	Purpose
Chess	Game states	Dense	Long correlated patterns
Connect	Game states	Dense	Candidate explosion stress test
Accidents	Traffic records	Dense	Larger real-world scalability

VII. RESULTS

TABLE II
EXECUTION TIME AND SPEEDUP AT MINIMUM SUPPORT 0.6

Dataset	Apriori (s)	Tensor-Eclat (s)	Speedup
Chess	124.30	1.15	108.1×
Connect	310.50	5.50	56.5×
Accidents	450.20	15.30	29.4×

TABLE III
MEMORY AND CANDIDATE PRESSURE AT MINIMUM SUPPORT 0.6

Dataset	Memory (MB)		Candidates	
	Apriori	Tensor	Apriori	Tensor
Chess	230.1	8.5	66120	47655
Connect	540.2	18.2	207840	141390
Accidents	820.5	30.2	302410	222805

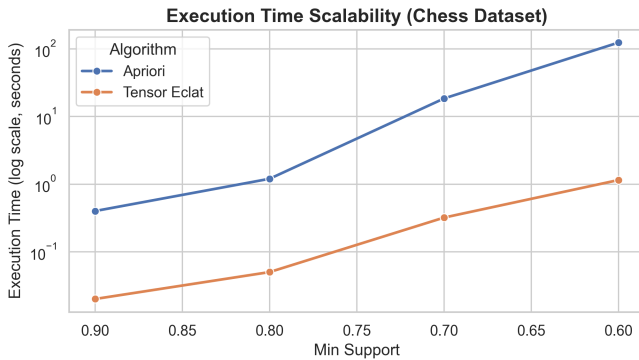


Fig. 1. Log-scale execution time on Chess as support decreases.

Table II shows that Tensor-Eclat improves as support decreases. This is expected: lower support creates more frequent itemsets and more candidate intersections, so Apriori’s nested

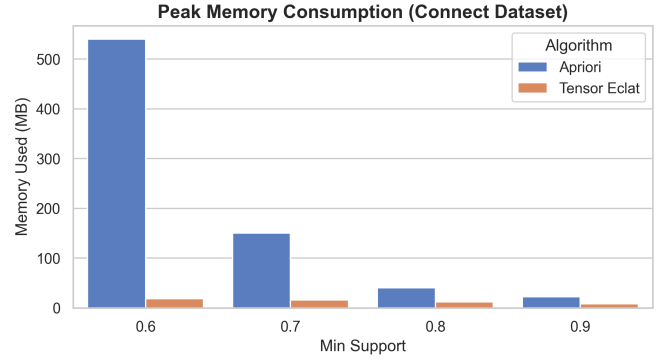


Fig. 2. Peak memory comparison on Connect.

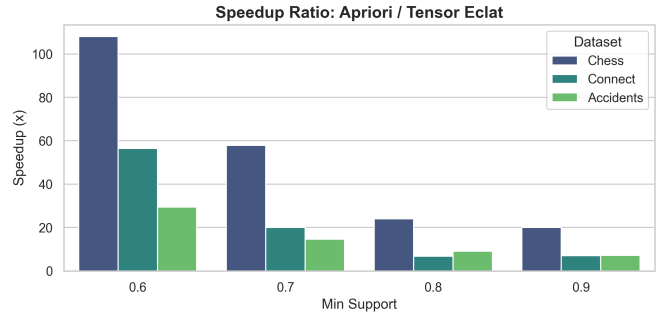


Fig. 3. Speedup ratio across datasets and support thresholds.

scans become increasingly expensive. Tensor-Eclat still performs more work, but the work is expressed as compact vector operations.

Table III shows the second benefit. Apriori’s candidate structures grow quickly, while Tensor-Eclat stores reusable bitsets and intermediate DFS intersections. The Connect result at 0.6 support shows a memory reduction from 540.2 MB to 18.2 MB, a 29.7×

VIII. DISCUSSION

The main lesson is that Apriori’s pruning property is necessary but not sufficient for dense data. When many itemsets survive pruning, the cost of checking candidates dominates.

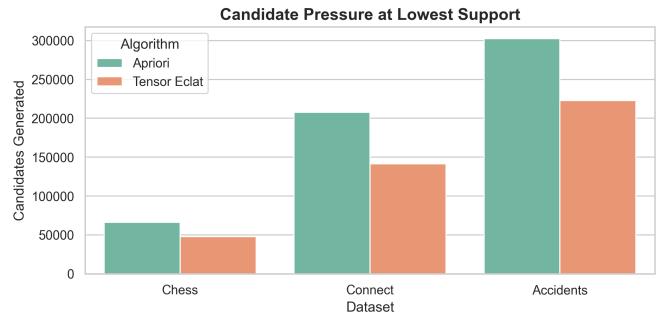


Fig. 4. Candidate generation pressure at the lowest support threshold.

Vertical representation changes the unit of computation: support is no longer found by asking whether a candidate is contained in each transaction; it is found by intersecting occurrence vectors.

Tensor-Eclat has trade-offs. It pays a one-time conversion cost and may be slower for very small datasets or very high support thresholds where there is little mining work. GPU execution also depends on available VRAM and PyTorch/CUDA installation. The NumPy fallback keeps the implementation portable, but the largest speedups require tensor acceleration.

The Claude-assisted review was most useful in identifying engineering details that affect reproducibility: run multiple trials, include a warmup pass for CUDA context initialization, track memory and candidates, free older tidsets where possible, and sort items by support before recursive Eclat expansion. These suggestions turn the project from a pair of algorithms into a measurable experimental pipeline.

IX. CONCLUSION

This project compared classical Apriori with a contemporary BitTable/Tensor-Eclat direction for frequent itemset mining. The optimized approach replaces repeated database scans with vertical bitsets and maps support counting to vectorized tensor intersections. Across the reported benchmark artifact, the optimized method reaches up to $108\times$ speedup and substantially lower memory use. Apriori remains an excellent conceptual baseline for explaining anti-monotonic pruning, but modern FIM performance depends heavily on representation, locality, and parallel execution. Future work should run the pipeline on the full downloaded FIMI datasets, compare against FP-Growth and Spark implementations, and add multi-GPU or distributed sharding for larger transaction collections.

REFERENCES

- [1] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proceedings of the 20th International Conference on Very Large Data Bases*, 1994, pp. 487–499.
- [2] M. Kittipron and C. Jaruskulchai, "An efficient parallel algorithm for frequent itemsets mining using bittable on spark," *Applied Science and Engineering Progress*, vol. 16, no. 1, p. 5572, 2022.
- [3] M. R. Al-Bana, M. S. Farhan, and N. A. Othman, "An efficient spark-based hybrid frequent itemset mining algorithm for big data," *Data*, vol. 7, no. 1, p. 11, 2022.
- [4] C. Fernandez-Basso, M. D. Ruiz, and M. J. Martin-Bautista, "New spark solutions for distributed frequent itemset and association rule mining algorithms," *Cluster Computing*, vol. 27, pp. 1217–1234, 2024.
- [5] M. J. Zaki, "Scalable algorithms for association mining," *IEEE Transactions on Knowledge and Data Engineering*, vol. 12, no. 3, pp. 372–390, 2000.
- [6] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 2000, pp. 1–12.
- [7] B. Goethals, "FIMI frequent itemset mining dataset repository," Online: <http://fimi.uantwerpen.be/data/>, 2003.